

TT-002: Esquema Prisma y Modelo de Datos

Responsable: Yandri Alcivar **Sprint:** 1 — Primera Impresión y Navegación **Estimación:** 5 puntos **Estado:**  Completado

Objetivo

Diseñar e implementar el modelo de datos completo del proyecto usando Prisma ORM, con 7 entidades que cubren autenticación, perfiles, catálogo de propiedades, geolocalización, galería de imágenes y mensajería.

Tecnologías instaladas

Paquete	Versión	Propósito
<code>prisma</code>	<code>^7.8.0</code>	CLI para migraciones y generar cliente
<code>@prisma/client</code>	<code>^7.8.0</code>	Cliente ORM para consultar la DB

Configuración

`prisma.config.ts`

```
import "dotenv/config";
import { defineConfig } from "prisma/config";

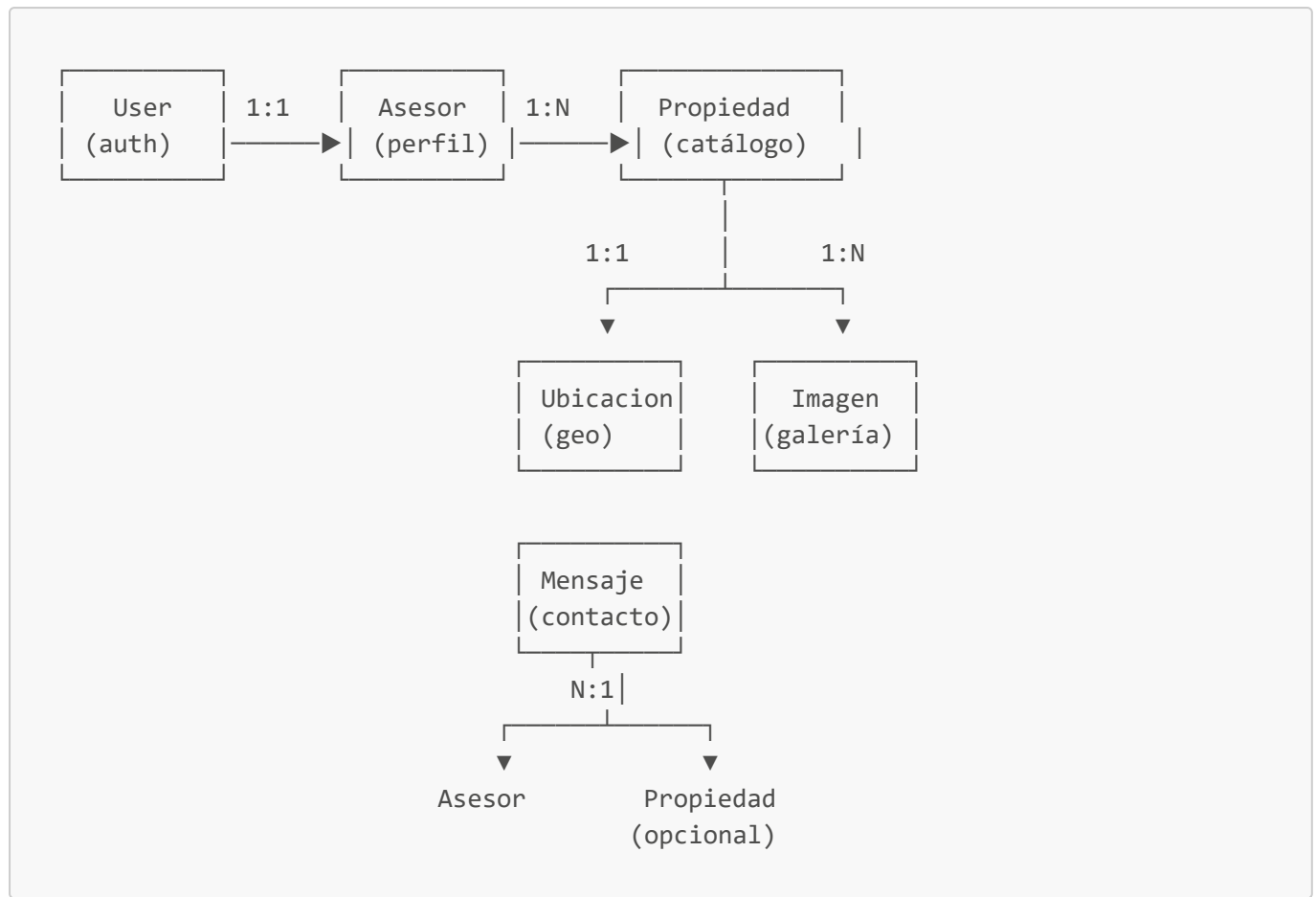
export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
  },
  datasource: {
    url: process.env["DATABASE_URL"],
  },
});
```

`.env` — Conexión a Supabase

```
DATABASE_URL="postgresql://postgres.gvoglwtbfmcporpbim:[YOUR-PASSWORD]@aws-1-us-east-1.pooler.supabase.com:6543/postgres"
```

Nota: Se usó el **Transaction Pooler** de Supabase (puerto 6543). La conexión directa (puerto 5432) se configuró en el archivo `.env` pero se usó la del pooler por compatibilidad con entornos serverless.

Diagrama de Entidades y Relaciones



Enums

Rol

```
enum Rol {  
  asesor  
  admin  
}
```

TipoPropiedad

```
enum TipoPropiedad {  
  casa  
  departamento  
  terreno  
  local  
  oficina  
}
```

TipoTransaccion

```
enum TipoTransaccion {  
  venta      # Propiedad en venta (el frontend lo muestra como "Comprar")  
  alquiler   # Propiedad en alquiler  
}
```

EstadoPropiedad

```
enum EstadoPropiedad {  
  activa  
  pausada  
  vendida  
  alquilada  
}
```

TipoMensaje

```
enum TipoMensaje {  
  contacto    # Consulta general sobre una propiedad  
  venta       # El cliente quiere vender su propiedad  
}
```

Entidades

1. User — Autenticación y roles

Campo	Tipo	Descripción
id	String @id @default(uuid())	UUID único
email	String @unique	Correo electrónico (login)
password	String	Hash bcrypt
rol	Role	asesor o admin
activo	Boolean @default(true)	Si la cuenta está activa
createdAt	DateTime @default(now())	Fecha de creación
updatedAt	DateTime @updatedAt	Fecha de actualización

Relación: Asesor? (1:1)

2. Asesor — Perfil profesional

Campo	Tipo	Descripción
id	String @id @default(uuid())	UUID único
userId	String @unique	FK → User.id
nombre	String	Nombre completo
telefono	String	Teléfono de contacto
foto	String?	URL de Cloudinary (opcional)
especialidad	String?	Ej: "Casas de lujo", "Departamentos"
descripcion	String?	Biografía profesional
añosExperiencia	Int?	Años de experiencia
createdAt	DateTime @default(now())	
updatedAt	DateTime @updatedAt	

Relaciones: User (1:1), Propiedad[] (1:N), Mensaje[] (1:N)

3. Propiedad — Catálogo de inmuebles

Campo	Tipo	Descripción
id	String @id @default(uuid())	UUID único
titulo	String	Título de la propiedad
descripcion	String	Descripción detallada
precio	Float	Precio en USD
tipoPropiedad	TipoPropiedad	Casa, depto, terreno, local, oficina
tipoTransaccion	TipoTransaccion	Venta o alquiler
habitaciones	Int?	Número de habitaciones
banios	Int?	Número de baños
parqueos	Int?	Número de parqueos
metrajeTotal	Float?	Metraje total en m²
metrajeConstruido	Float?	Metraje construido en m²
estado	EstadoPropiedad @default(activa)	Estado actual
destacada	Boolean @default(false)	Si aparece en destacados
asesorId	String	FK → Asesor.id
ubicacionId	String @unique	FK → Ubicacion.id
createdAt	DateTime @default(now())	

Campo	Tipo	Descripción
updatedAt	DateTime @updatedAt	

Relaciones: Asesor (N:1), Ubicacion (1:1), Imagen[] (1:N), Mensaje[] (1:N)

4. Ubicacion — Geolocalización

Campo	Tipo	Descripción
id	String @id @default(uuid())	UUID único
direccion	String	Dirección completa
sector	String?	Barrio o sector
ciudad	String @default("Guayaquil")	Ciudad
provincia	String @default("Guayas")	Provincia
latitud	Float	Coordenada latitud
longitud	Float	Coordenada longitud

Relación: Propiedad? (1:1)

5. Imagen — Galería Cloudinary

Campo	Tipo	Descripción
id	String @id @default(uuid())	UUID único
url	String	URL de Cloudinary
publicId	String	ID público en Cloudinary
orden	Int @default(0)	Orden de visualización
propiedadId	String	FK → Propiedad.id

Relación: Propiedad (N:1)

6. Mensaje — Contacto y solicitud de venta

Campo	Tipo	Descripción
id	String @id @default(uuid())	UUID único
nombre	String	Nombre del contacto
email	String	Correo del contacto
telefono	String?	Teléfono (opcional)
mensaje	String	Contenido del mensaje
tipo	TipoMensaje	contacto o venta

Campo	Tipo	Descripción
leido	Boolean @default(false)	Si el asesor lo leyó
archivado	Boolean @default(false)	Si está archivado
propiedadId	String?	FK → Propiedad.id (opcional)
asesorId	String	FK → Asesor.id
createdAt	DateTime @default(now())	
updatedAt	DateTime @updatedAt	

Relaciones: `Propiedad?` (N:1 opcional), `Asesor` (N:1)

Schema Prisma completo

El archivo completo está en `prisma/schema.prisma` (137 líneas) y define:

- **5 enums:** `Rol`, `TipoPropiedad`, `TipoTransaccion`, `EstadoPropiedad`, `TipoMensaje`
- **7 modelos:** `User`, `Asesor`, `Propiedad`, `Ubicacion`, `Imagen`, `Mensaje`
- **Cliente generado** en `src/generated/prisma/`

Comandos ejecutados

```
# 1. Instalar Prisma
cd apps/backend
npm install prisma @prisma/client

# 2. Inicializar Prisma
npx prisma init

# 3. Configurar .env con conexión a Supabase
# (se editó manualmente con las credenciales del Transaction Pooler)

# 4. Editar prisma/schema.prisma con las 7 entidades
# (se escribieron 137 líneas de schema)

# 5. Validar y generar el cliente
npx prisma validate
npx prisma generate

# 6. Subir a GitHub con feature branch
git checkout -b feature/yandri-tt-002-prisma
git add .
git commit -m "TT-002: esquema Prisma y modelo de datos (7 entidades)"
git push origin feature/yandri-tt-002-prisma
# PR creado en GitHub y mergeado a main
```

Verificación

```
npx prisma validate
# Output: The schema at prisma/schema.prisma is valid 🚀

npx prisma generate
# Output: ✓ Generated Prisma Client (7.8.0) to .\src\generated\prisma in 79ms
```

Decisiones de diseño

¿Por qué User + Asesor separados?

- **Principio SRP (Single Responsibility):** **User** maneja solo autenticación (email, password, rol); **Asesor** maneja solo el perfil profesional (nombre, foto, especialidad)
- **Admin sin datos huérfanos:** un admin solo tiene **User { rol: 'admin' }**, sin necesidad de crear un perfil Asesor con campos null
- **Flexibilidad futura:** si después se necesitan más roles o perfiles, la separación ya está hecha

¿Por qué solo 'venta' | 'alquiler' en TipoTransaccion?

- **"Comprar"** es una etiqueta del frontend que filtra propiedades con **tipoTransaccion: 'venta'**
- En la base de datos, la propiedad está **"en venta"** no "en compra"
- Esto sigue el mismo modelo que RE/MAX Ecuador y portales inmobiliarios reales

¿Por qué Mensaje sirve para contacto y para solicitud de venta?

- Ambos casos comparten los mismos campos (nombre, email, teléfono, mensaje)
- Se diferencian con el campo **tipo: 'contacto' | 'venta'**
- La solicitud de venta no necesita tabla separada, evitando duplicación

Próximos pasos

- **TT-003:** Crear las tablas en Supabase con **prisma migrate dev --name init** y conectar Express a Prisma
- **TT-004:** CRUD de Propiedades (controllers, services, repositories, DTOs, rutas)
- **TT-005:** CRUD de Asesores
- **TT-006:** API de Mensajes + subida de imágenes a Cloudinary